

corresponds to a lower point in the terrain. When we apply the `perturb()` function, we're moving to a different point in the terrain, hoping that point is as high as possible.

We'll use the temperature function to guide our exploration of this terrain. When we start, our high temperature will dictate more openness to choosing a worse itinerary. Closer to the end of the process, we'll be less open to choosing worse itineraries and more focused on "greedy" optimization.

The algorithm we'll implement, *simulated annealing*, is a modified form of the `perturb` search algorithm. The essential difference is that in simulated annealing, we're sometimes willing to accept itinerary changes that *increase* the distance traveled, because this enables us to avoid the problem of local optimization. Our willingness to accept worse itineraries depends on the current temperature.

Let's modify our `perturb()` function with this latest change. We'll add a new argument: `time`, which we'll have to pass to `perturb()`. The `time` argument measures how far we are through the simulated annealing process; we start with `time` 1 the first time we call `perturb()`, and then `time` will be 2, 3, and so on as many times as we call the `perturb()` function. We'll add a line that specifies the temperature function and a line that selects a random number. If the random number is lower than the temperature, then we'll be willing to accept a worse itinerary. If the random number is higher than the temperature, then we won't be willing to accept a worse itinerary. That way, we'll have occasional, but not constant, times when we accept worse itineraries, and our likelihood of accepting a worse itinerary will decrease over time as our temperature cools. We'll call this new function `perturb_sa1()`, where `sa` is short for simulated annealing. [Listing 6-4](#) shows our new `perturb_sa1()` function with these changes.

```
def perturb_sa1(cities, itinerary, time):
    neighborids1 = math.floor(np.random.rand() * (len(itinerary)))
    neighborids2 = math.floor(np.random.rand() * (len(itinerary)))

    itinerary2 = itinerary.copy()

    itinerary2[neighborids1] = itinerary[neighborids2]
    itinerary2[neighborids2] = itinerary[neighborids1]
```